ScanSuite
User Manual

Version 0.4 Draft 01.03.2024

Author: Sergey Egorov

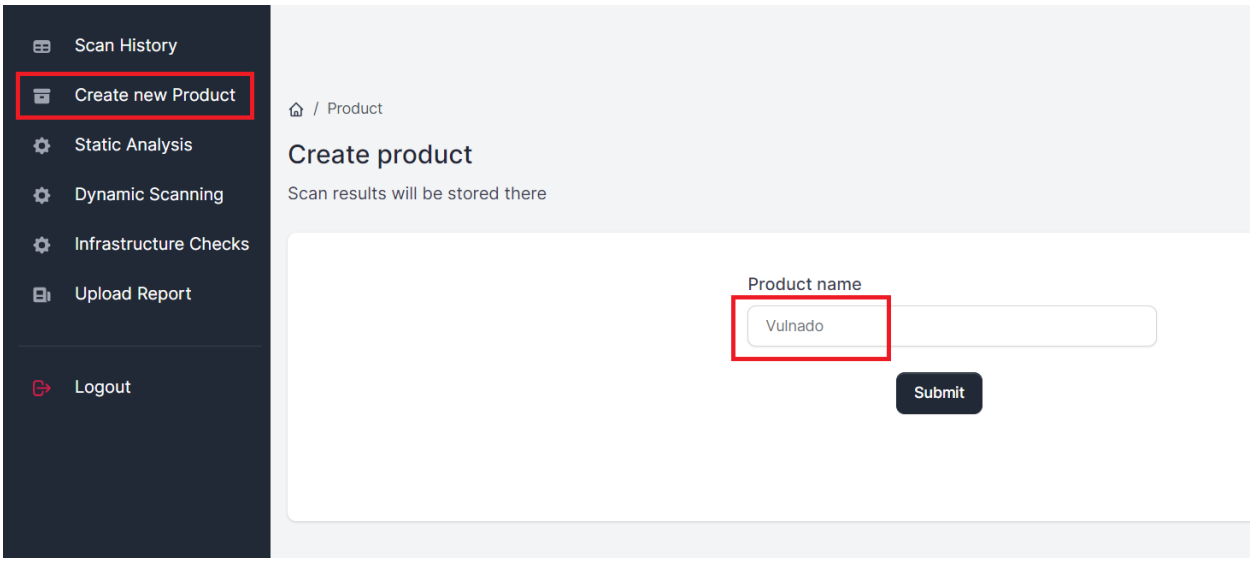
Contents

Performing a static code analysis 3
Following the scan execution 5
Executing the OpenAI scan 7
Compiling the code (only for Java and C#) 8
Working with results 10
Performing dynamic web scanning 13
Running infrastructure tests 16
Creating own scanning rules 17

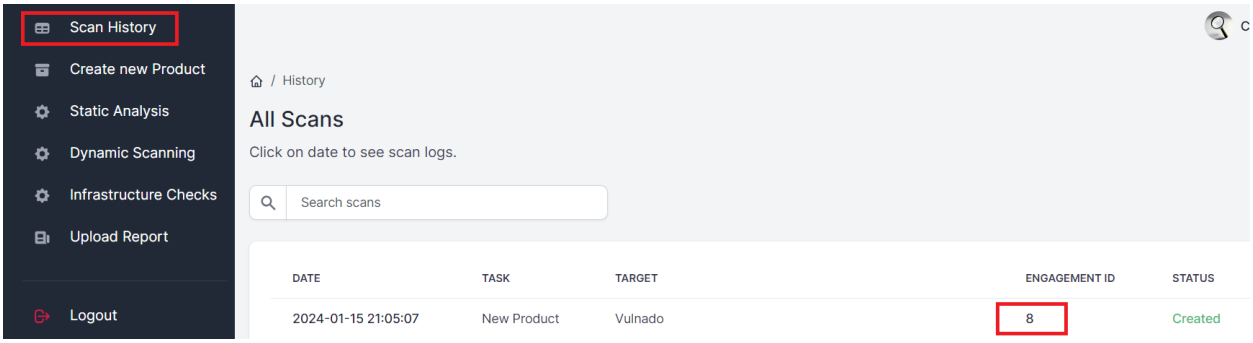
Performing a static code analysis

Static code analysis includes invocation of SASTs, Secrets scans, Dependency checks and Infrastructure as Code scanners.

To do that, **first, create a new product**, where scan results will be stored. Give it a unique name, it will not be possible to create 2 products with the same name:



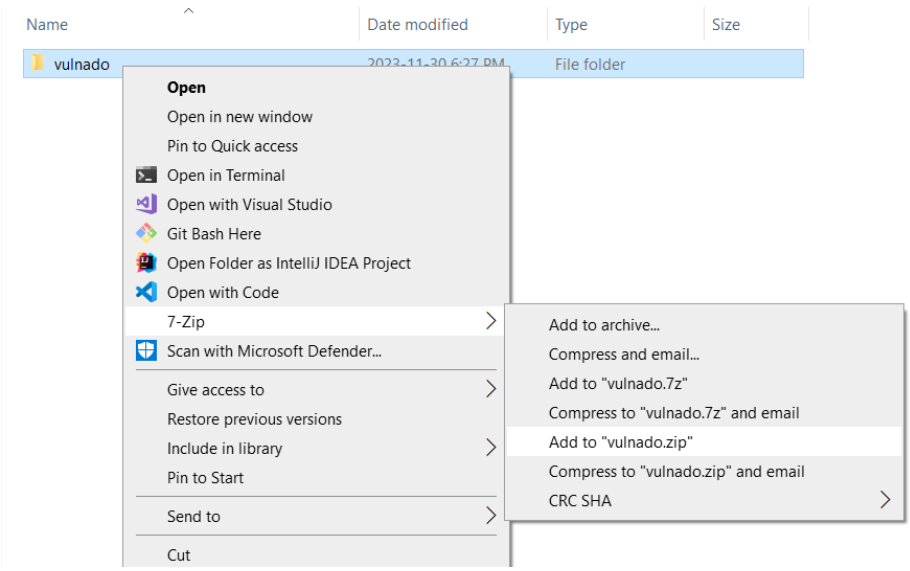
Once the product is created, the respective Engagement ID can be found in the **Scan History** tab. This ID will be used as a reference for the scan results storage in DefectDojo:



Also, this ID will be saved and auto filled in the scanning forms so no need to memorize it all the time.

Now, prepare the ZIP file which includes the folder with the source code.

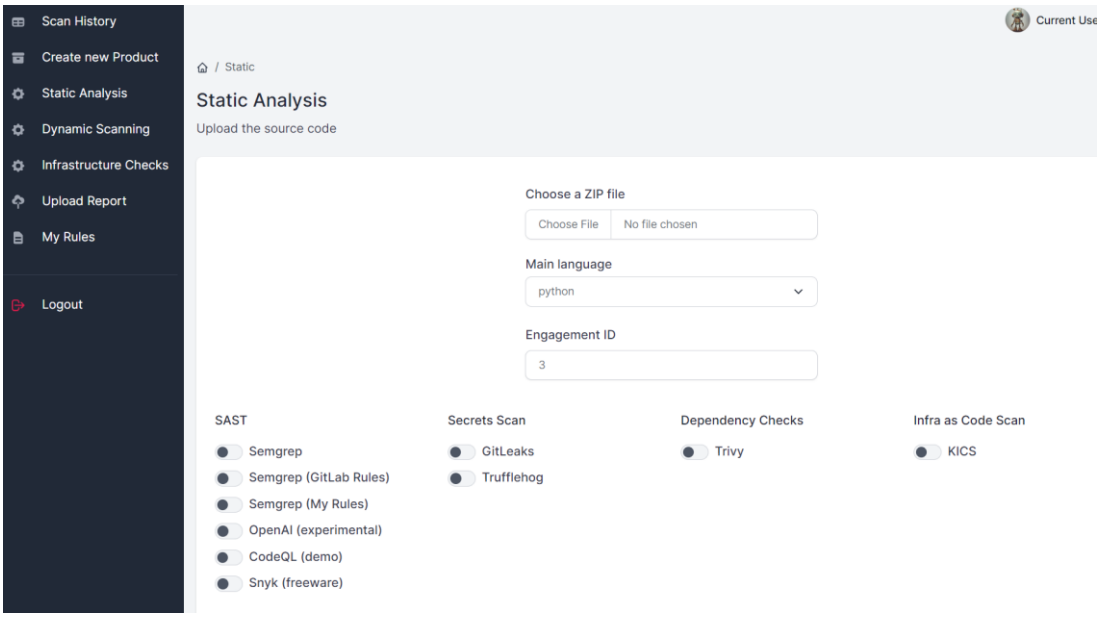
Important: Copy all source code files inside the folder before archiving, as the ScanSuite will search for folders after unzipping:



Go to the **Static Analysis** tab and upload the ZIP archive. Choose the main language and multi-language scanners you want to be invoked.

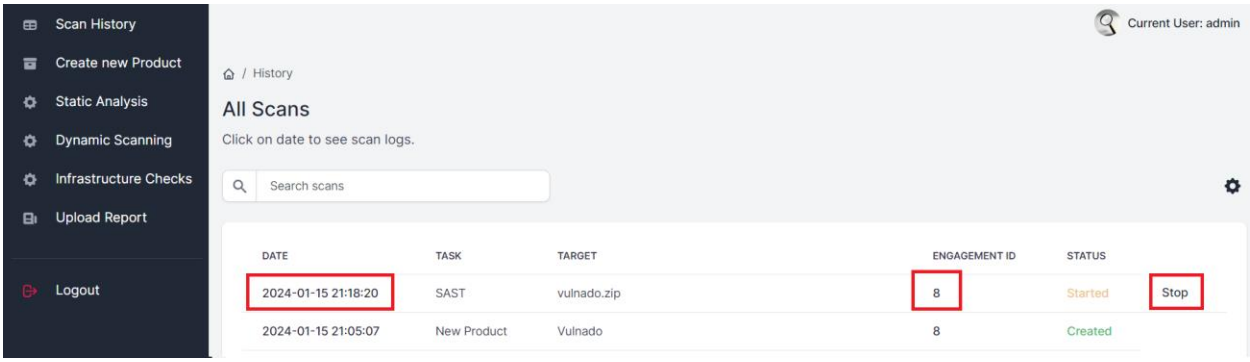
<u>Language specific scanners will also be executed</u> based on the specified "Main language" field.

Double check the Engagement ID, should be the same as created with product, and hit "Upload":



Following the scan execution

The scanning process can be monitored in the **Scan History** tab:

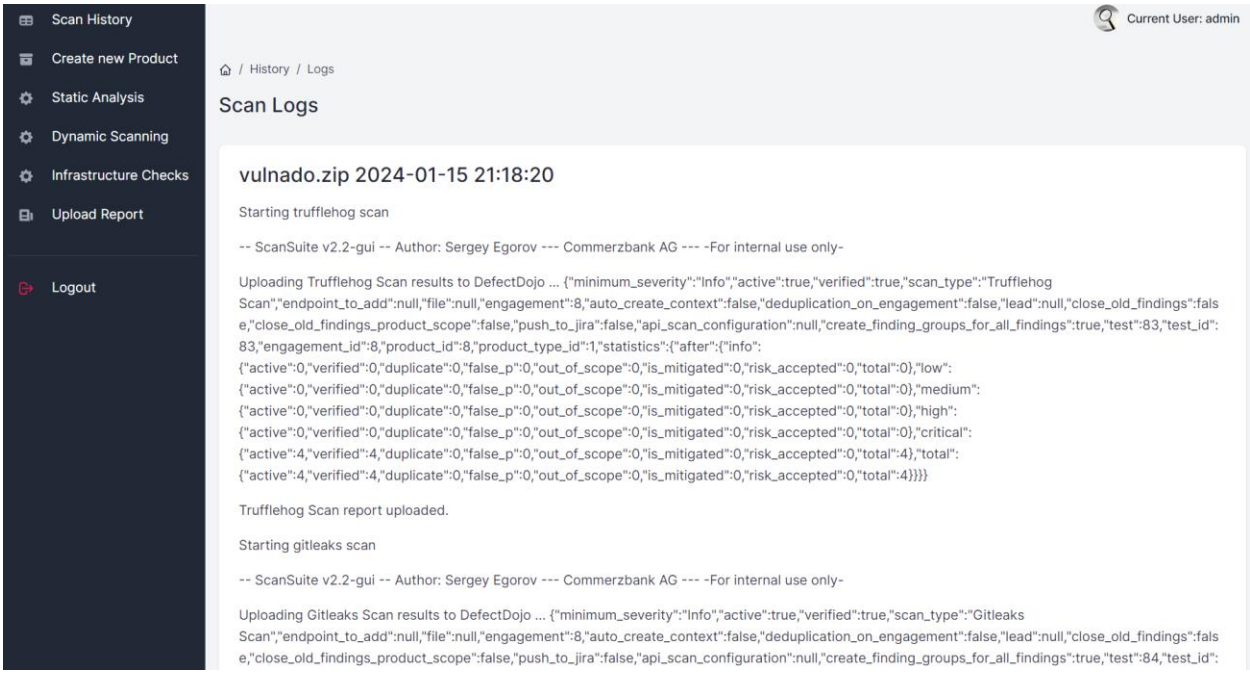


The scan **Status** is provided in the respective field.

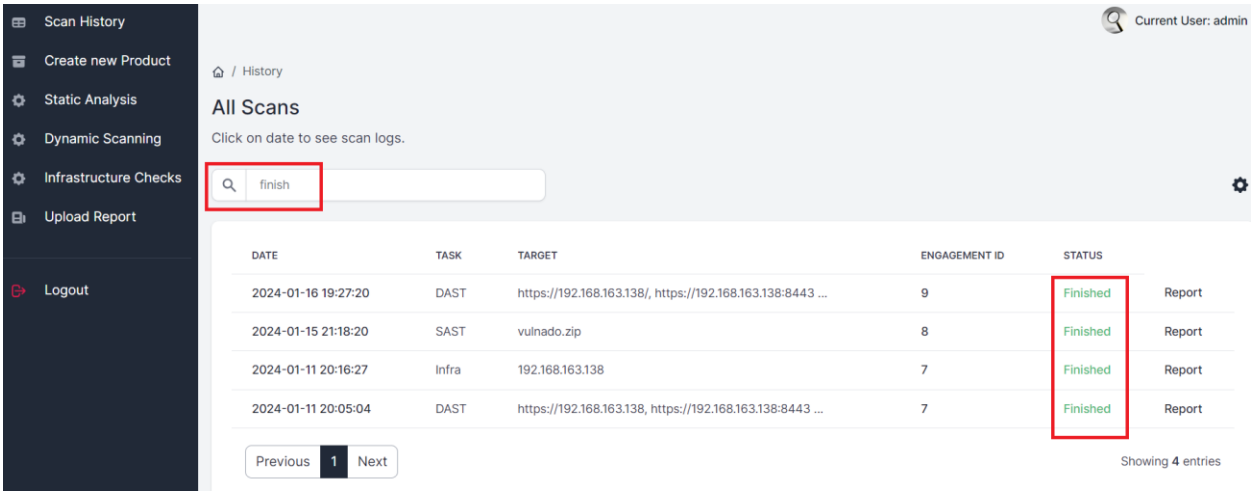
To cancel the scan, click the **Stop** button.

To see the **Results** in DefectDojo click on the respective Engagement ID number.

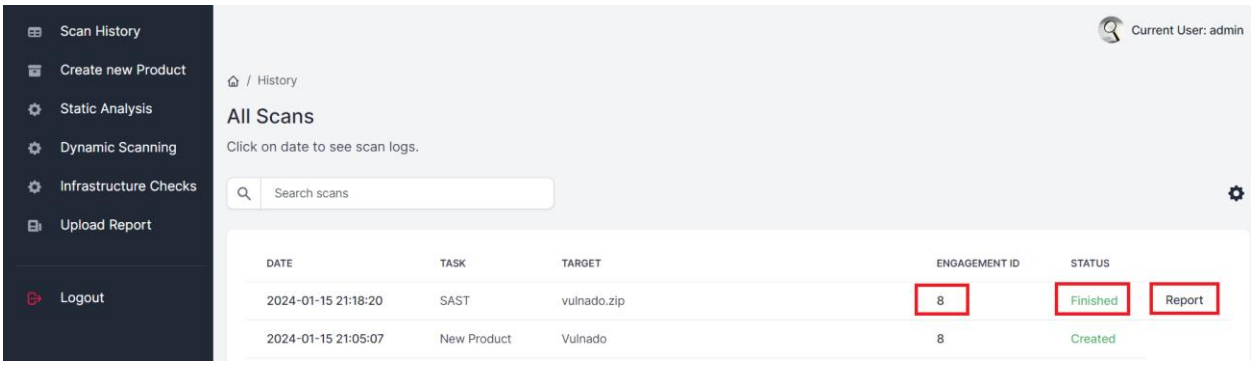
By clicking on the **Date** check the execution logs coming in the real time:



The search bar allows to search through all search fields, except the Date field:

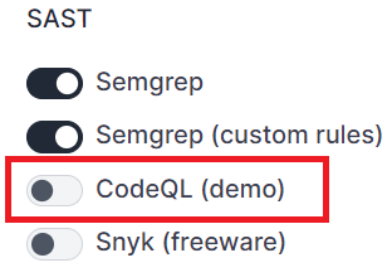


Once the scan is finished the status will be set as Finished. It is possible to download the raw report files by clicking on the **Report** button. However, this is not an intended way to work with the results and should be used for exceptional/debugging cases (unless you enjoy reading json/xml structures).



Important: CodeQL (SAST) has licensing limitation and is allowed be used for open-source projects or demo purposes.

Therefore, it is marked as "demo" and disabled by default. By invoking the scanner, you accept respective Terms & Conditions.



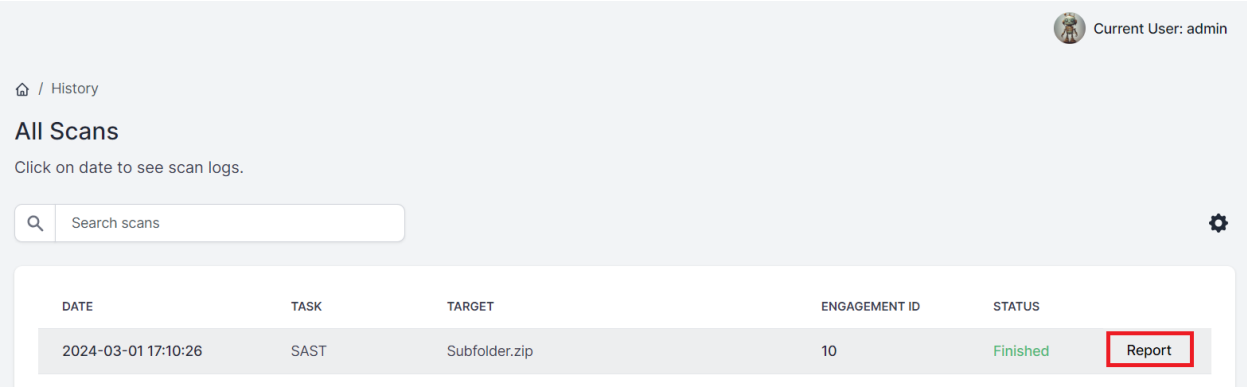
Executing the OpenAI scan

ScanSuite supports code analysis via the LLM / GenAI services, such OpenAI. Some considerations to mention:

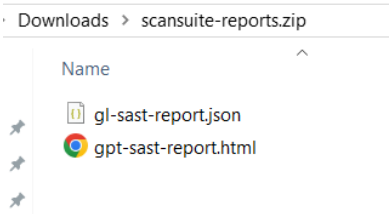
- The code will be submitted to OpenAI for analysis which may affect the code confidentiality.
- Analysis usually takes longer than other types of scans.
- Due to the speed and cost reasons, it is not recommended to upload all project files, such as configurations and dependency libraries, but only the ones which implement the core application logic.
- Results may vary for each upload as the analysis is done "on the fly" by OpenAI.

To set up an OpenAI scan you'd need to specify the key in the Settings menu. Refer to the Admin Manual on how to obtain and set this key.

Results are not uploaded to Defect Dojo for a few reasons. Instead, the html file is formed from the analysis results, which could be downloaded via the Report button after the scan is finished:



Download the report and open the **gpt-sast-report** HTML file:



Compiling the code (only for Java and C#)

Some Java and C#/.NET scanners, such as CodeQL, FindSecBugs, Security Code Scan etc. need to "instrument" the code (process similar to compilation) before analyzing it. This might be challenging as the source code should be complete and accurate, dependencies should be available offline, or it should be possible to download them either from internal artifactory or from the Internet.

Please consult the developers or your teammates for possible ways of obtaining compileable version of the code as the scanning results may be worth this extra mile.

Here we will look at the case where Maven is used to build Java projects.

This build system, along with Gradle, Ant and a few more are supported by the SAST scanners, such as CodeQL and FindSecurityBugs, which will attempt to download the dependencies from locations, mentioned in pom.xml and compile the code for further analysis.

This works well if the dependency repository is reachable by the scanner and does not require authentication to download modules, for example via the Internet.

If this is not the case, one can download dependencies beforehand and pass the cached modules along with the source code. Manually caching Maven dependencies is possible with the following command, executed from the source code folder:

./mvnw dependency:go-offline

```
root@cd818fdd3842:/src# ./mvnw dependency:go-offline
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.scalesec:vulnado >------------------------
[INFO] Building vulnado 0.0.1-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] >>> maven-dependency-plugin:3.1.1:go-offline (default-cli) > :resolve-plugin
s @ vulnado >>>
[INFO]
[INFO] --- maven-dependency-plugin:3.1.1:resolve-plugins (resolve-plugins) @ vulnad
o ---
[INFO]
[INFO] The following plugins have been resolved:
[INFO]    org.apache.maven.plugins:maven-jar-plugin:maven-plugin:3.1.1:runtime
[INFO]       org.apache.maven.plugins:maven-jar-plugin:jar:3.1.1
```

Dependencies will be downloaded and stored in **~/.m2** folder:

```
root@cd818fdd3842:/src# ls ~/.m2/
repository  wrapper
```

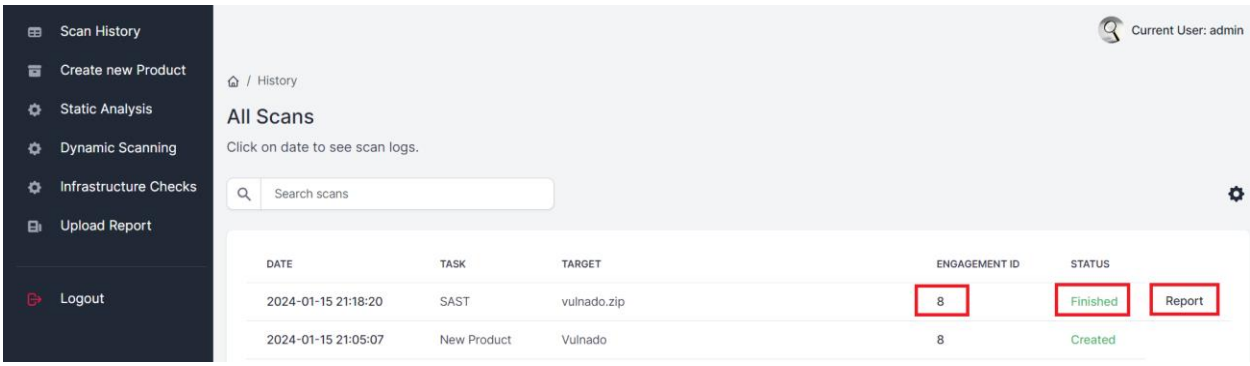
Both folders contain downloaded modules which could be provided to the offline ScanSuite installation. Copy them to the **dependency** folder of the source code and upload the code ZIP archive via web interface as usual:

```
scanbot@scansuite:~/vulnado$ ls
Dockerfile  client              exercises      mvnw      reverse_shell
LICENSE     dependency          foobar         mvnw.cmd  src
README.md   docker-compose.yml  internal_site  pom.xml   target
scanbot@scansuite:~/vulnado$ ls dependency/
repository  wrapper
```

Working with results

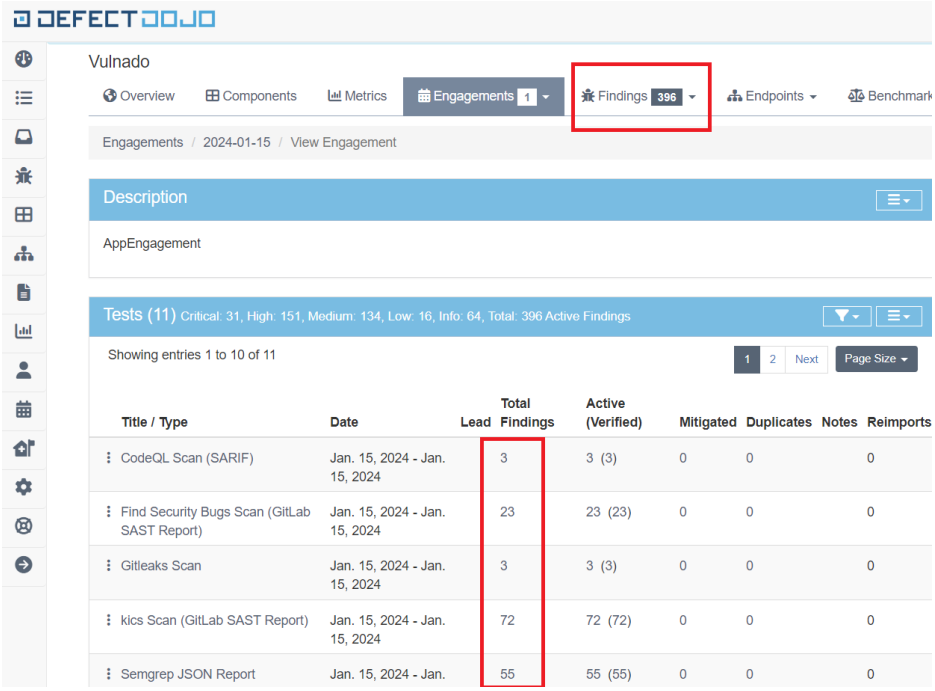
DefectDojo provides an opportunity to visualize the findings, process, track them, gather statistics, create reports, upload to Jira and many more.

In ScanSuite click on the **Engagement ID** number (8 on the screenshot below) to open results in DefectDojo:



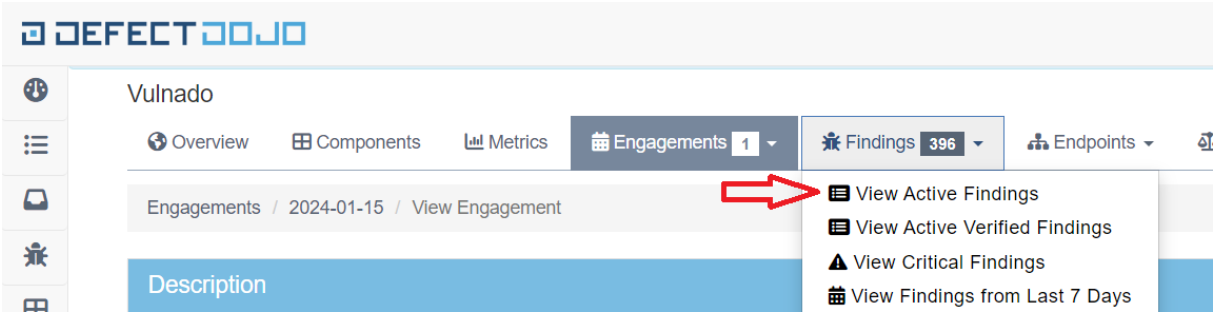
It will redirect you to the DefectDojo instance, installed on the same host. First time you will need to login to DefectDojo with provided credentials.

Once you have logged in, the Product -> Engagement view will be opened by the reference from ScanSuite so you do not need to search it within DefectDojo:

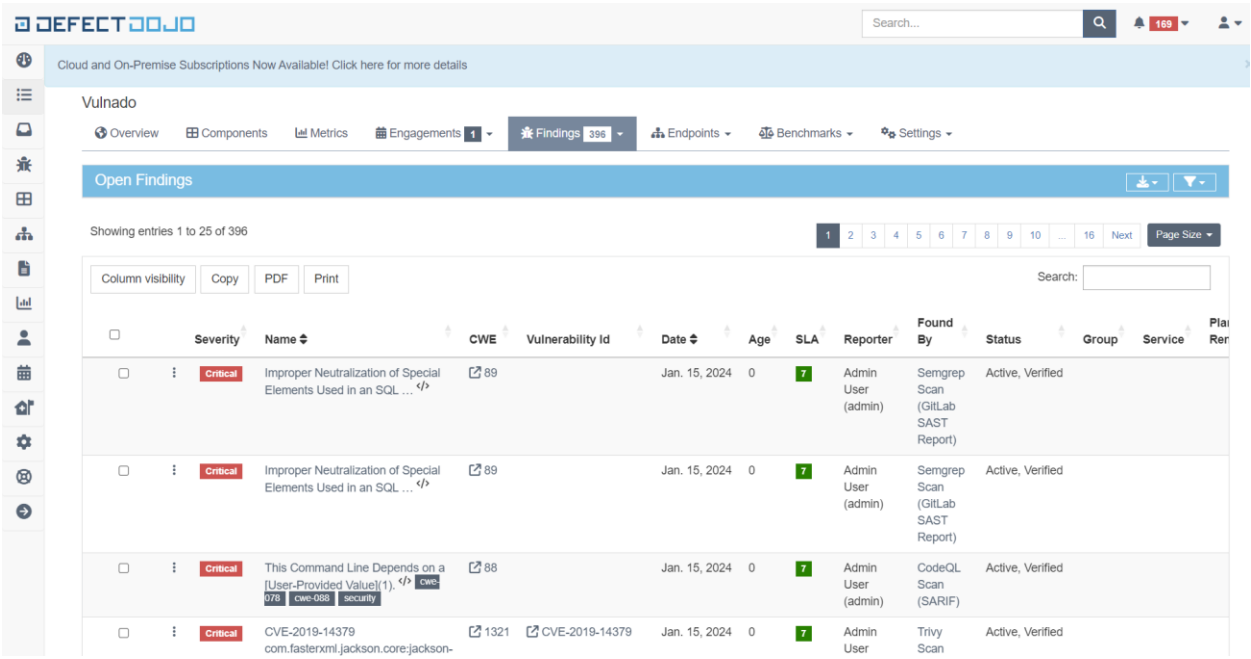


In this **Engagement** you will find all successful scan exports with possibilities to view the consolidated list of the findings or the ones which came from each individual scan.

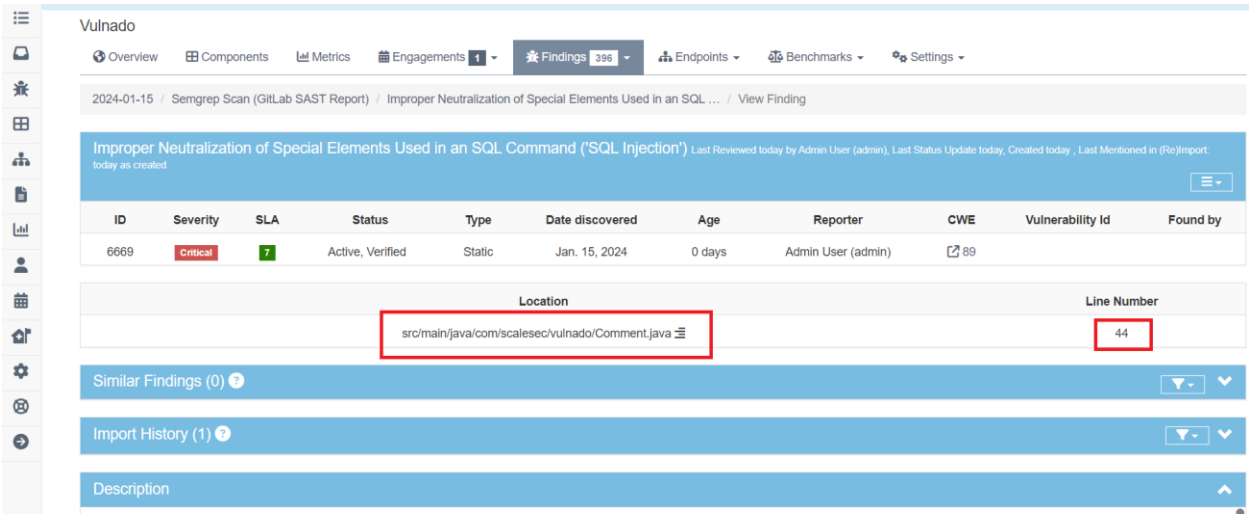
Open the consolidated view:



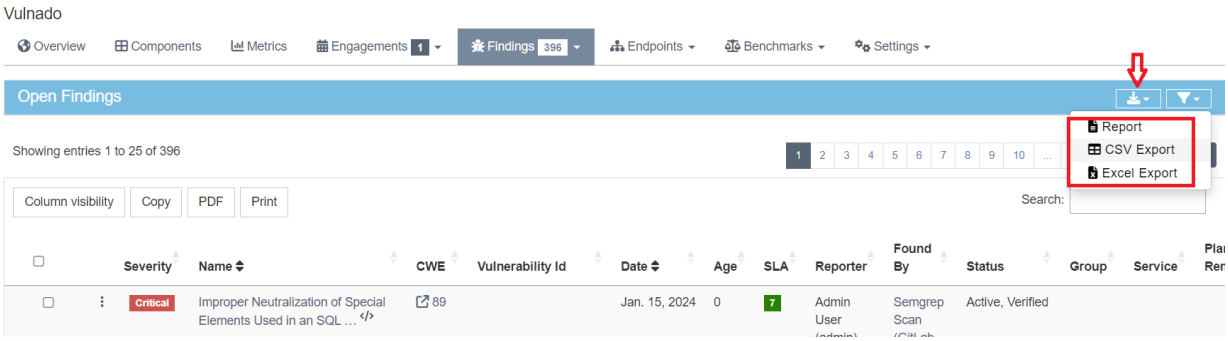
Here are all findings with the testing related data:



By clicking on each finding we get the details of it. These usually vary depending on the scanner, but most provide the data such as the affected file, containing vulnerabilities, line number for SAST, URL for DAST and some details in the Description and Mitigation sections of the finding.



Findings could also be exported in the HTML or CSV views:



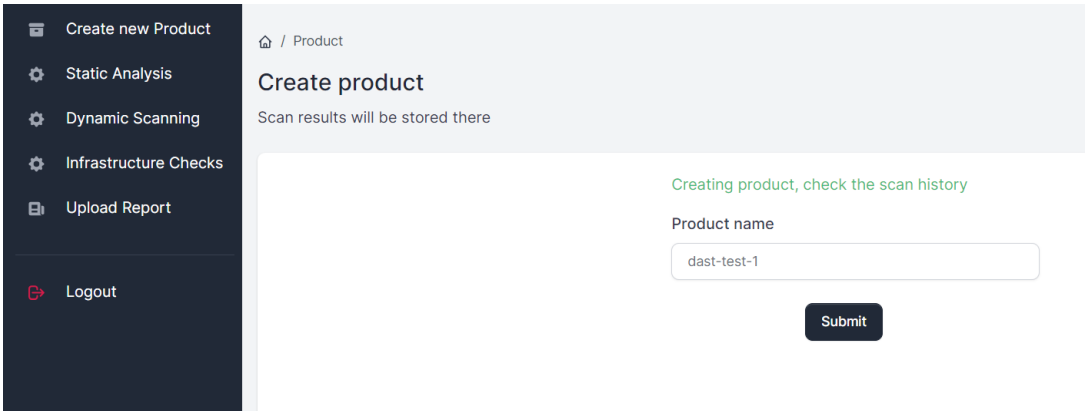
HTML report gives you all findings details in the nice and scrollable form. Sometimes it might be a preferred way to analyze the findings, rather than clicking on each one individually.

To learn more about Defect Dojo functionality, settings, and troubleshooting, refer to the original documentation at https://defectdojo.github.io/django-DefectDojo/

Performing dynamic web scanning

ScanSuite supports both authenticated and non-authenticated web scanning modes with various scanners.

It is recommended to create separate products for different scan types, and maybe even website groups as it will be easier to examine the results later:

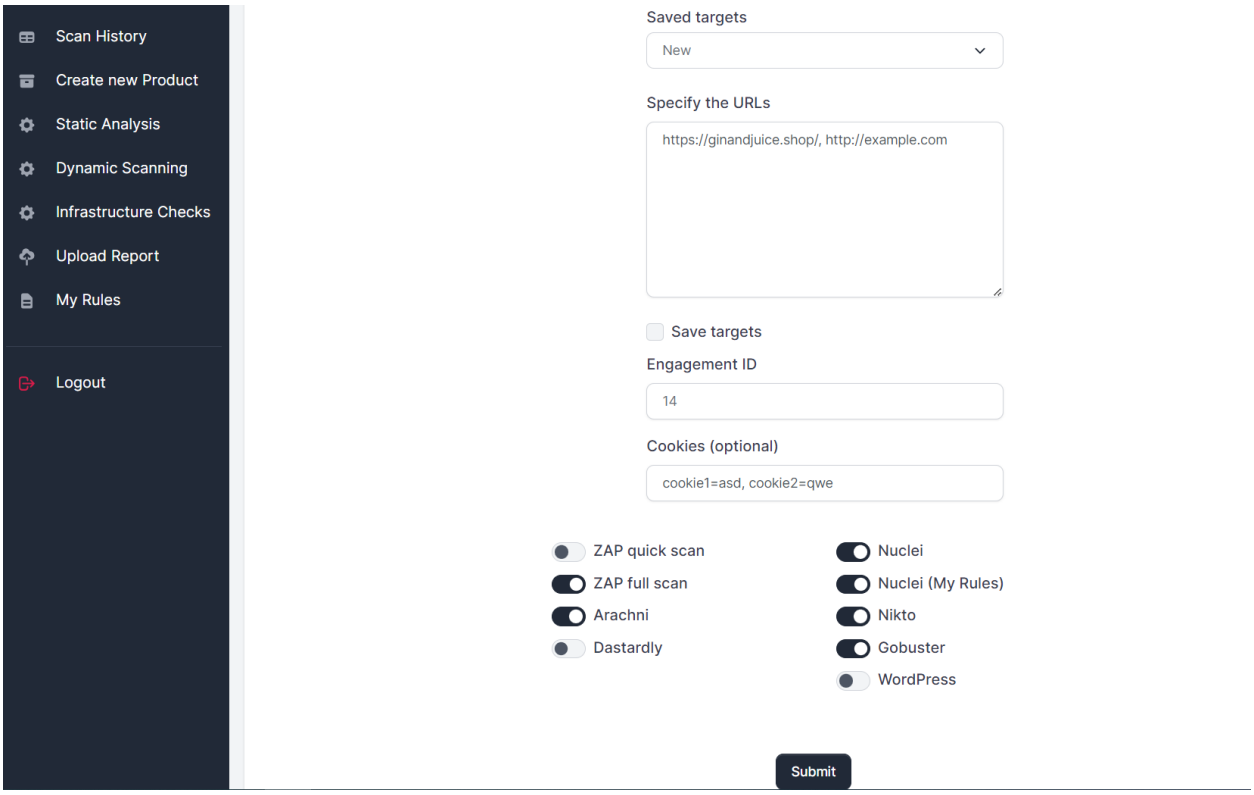


To run the web scan, follow the Dynamic scanning tab, enter the comma separated list of the URLs, choose the scanners you want to invoke.

Tick **Save targets** if you plan to reuse the same URL list in future. These will be available via **Saved targets** top dropdown list next time for the quick load.

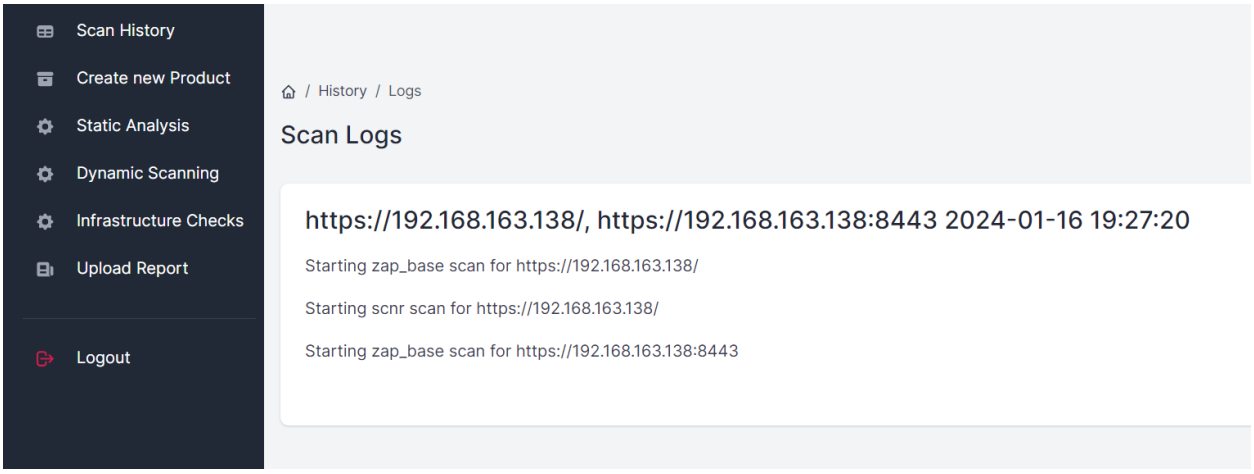
Most scanners, excluding Dastardly, support authenticated scans using provided **cookies**. To invoke such scan, enter comma **,** or semi-colon **;** separated cookies list.

As usual, ensure the right Engagement ID number is set so the results will be saved within the intended product folder and click Submit:



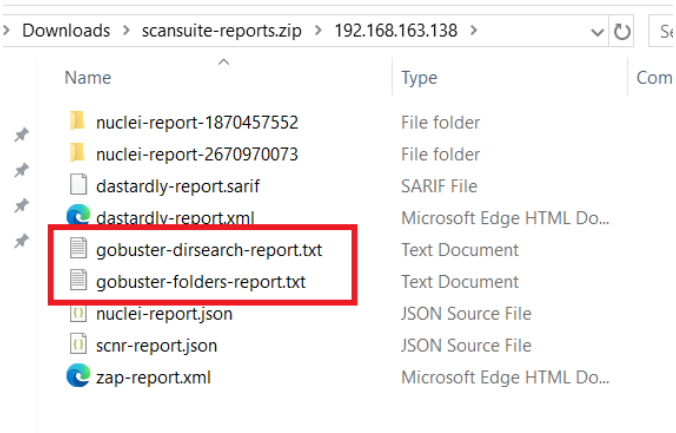
ScanSuite supports the concepts of distributed workers which will take each scanning task and execute it. The number of workers depends on the amount of time-consuming tasks, hardware capabilities (some scanners are resource consuming) and can be easily extended as required.

As depicted below on the History – Logs screenshot, several scans were started almost simultaneously, meaning 3 scanning workers are up and processing the tasks:



Once all targets are scanned by all chosen scanners, the scanning will be marked as Finished:

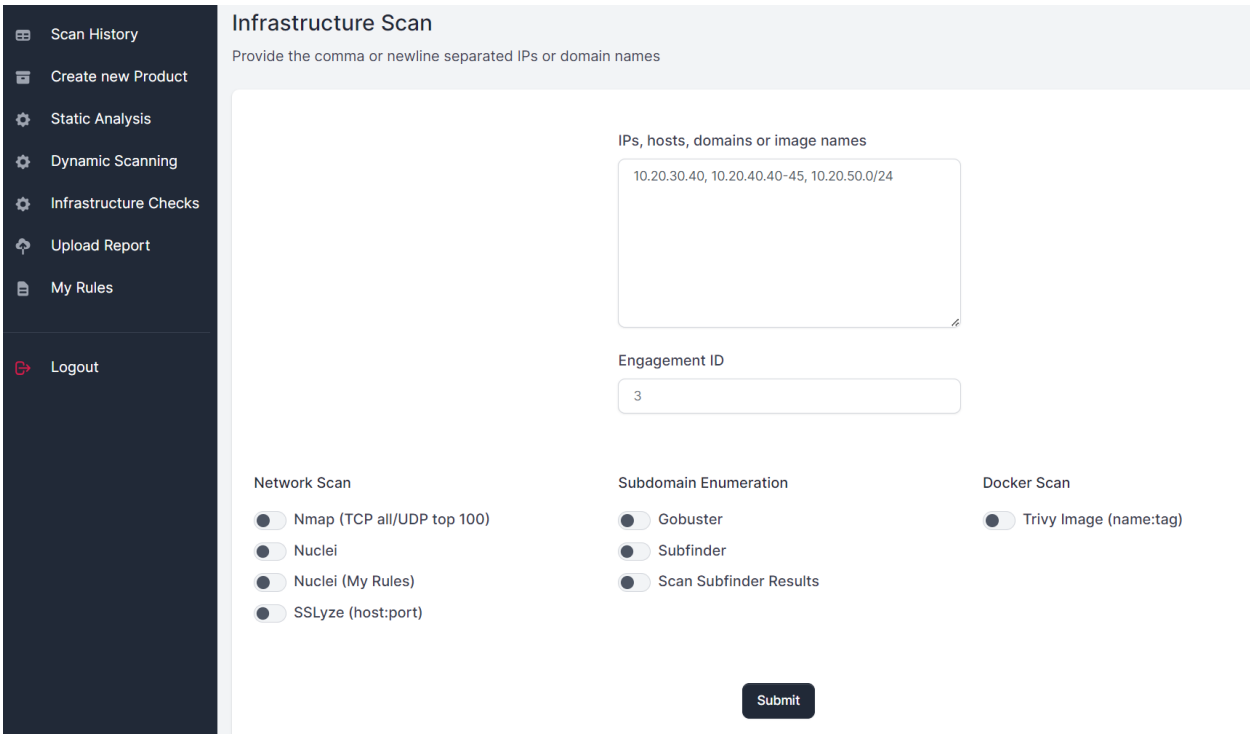
Note: Dirbusting / Gobuster results will not be exported to DefectDojo because of the nature of the scan. The best way to view revealed paths is by checking the scan logs or to download the Report by the end of the scan and check the scan reports:



Running infrastructure tests

ScanSuite supports several infrastructure checks which aim to extend the regular infrastructure scans functionality. There are currently 3 groups of checks available:

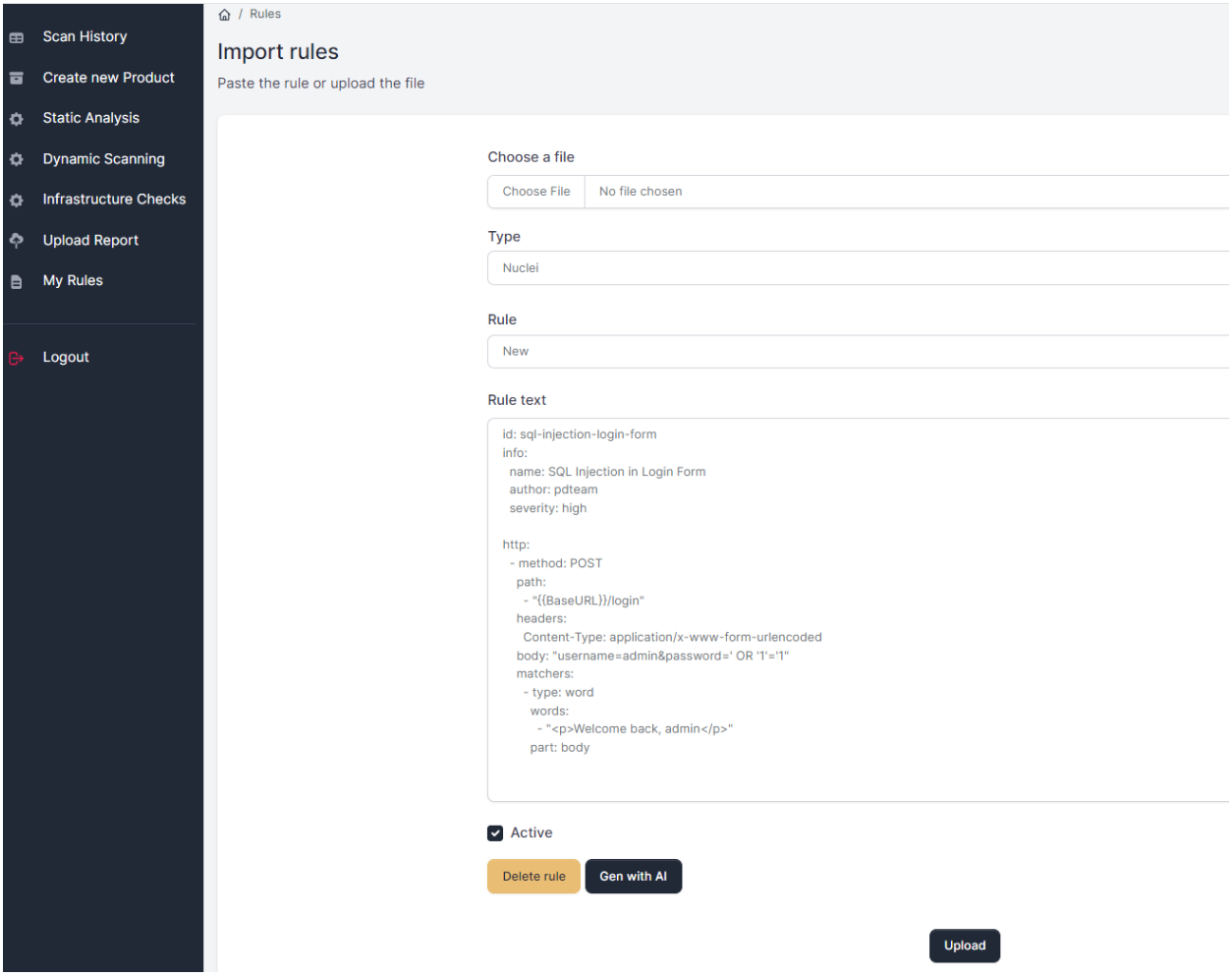
- **Network Scan** includes scanners to be run against the set of IPs or hostnames to identify open ports, listening services and their vulnerabilities.
- **Subdomain enumeration** which discovers subdomains for given domain names and checks them for vulnerabilities. The list of discovered subdomains will be accessible via the Report button once the scan is finished.
- **Docker Scan** identifies local packages vulnerabilities in running docker images.



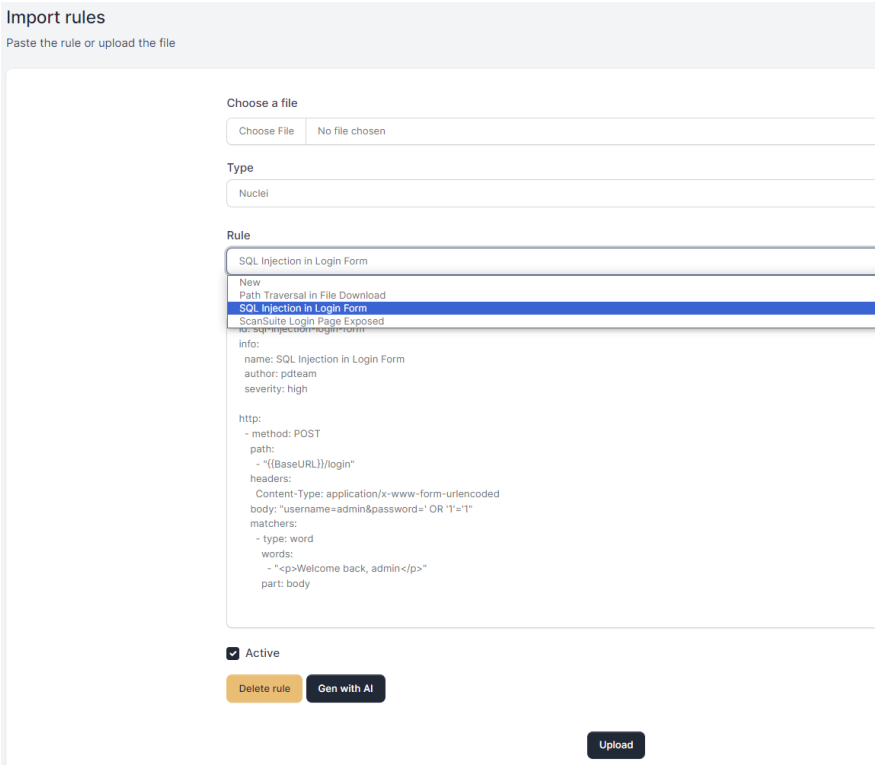
Creating own scanning rules

ScanSuite supports own set of scanning rules for both static (Semgrep) and dynamic (Nuclei) analysis.

To create a new rule, select the rule type (Nuclei or Semgrep), paste the rule text, and click upload:

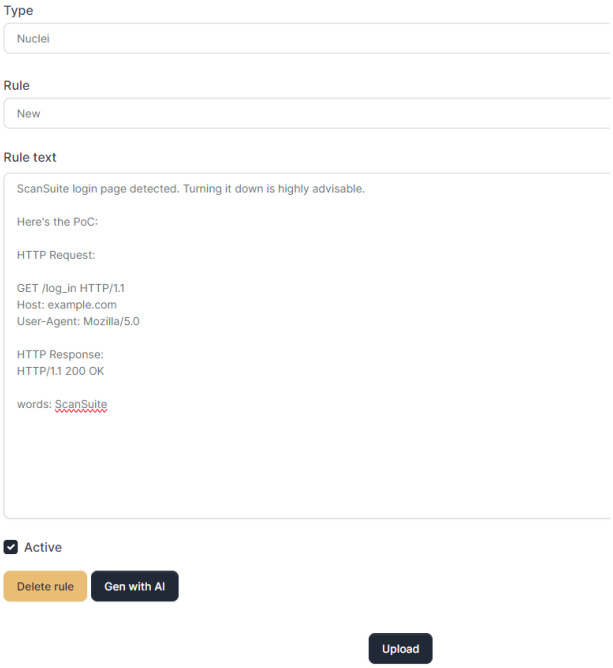


To view or change the rule select it via the drop-down menu:



It is possible to generate the Nuclei rule from the vulnerability PoC description using OpenAI connector. Refer to the Admin Manual to obtain and set up the **OpenAI API key**.

Paste the description including parts of the HTTP request and response used for the vulnerability detection and clock Gen with AI:



The rule will be generated and pasted to the same field. Review it, amend if necessary and click Upload:

Rule text

```
id: scansuite-login-page
info:
  name: ScanSuite Login Page Detected
  author: assistant
  severity: low
  description: |
    The login page of ScanSuite has been detected. It is recommended to turn it down to avoid potential vulnerabilities and im
  remediation: |
    Disable or remove the ScanSuite login page if not necessary for your application. Ensure proper access controls and authe
in place.
http:
  - method: GET
    path:
      - "{{BaseURL}}/log_in"
    matchers:
      - type: word
        words:
          - "ScanSuite"
        part: body
```

☑ Active

Delete rule | Gen with AI

Upload

Semgrep rules could be managed the same way, but the AI rule generation is not yet supported.

Choose a file

Choose File | No file chosen

Type

Semgrep

Rule

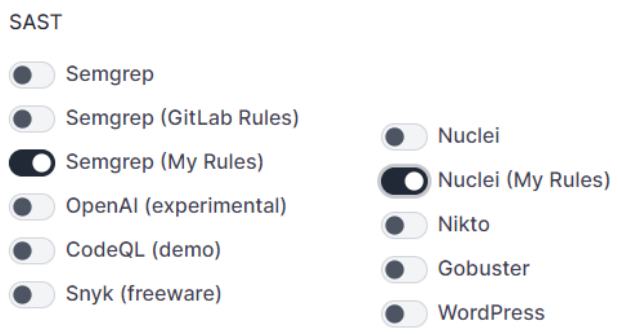
New

Rule text

```
rules:
- id: ts-firebase-api-key
  patterns:
    - pattern-either:
      - pattern: |
          const $A = { apiKey: '...' }
      - pattern: |
          const $A = await $B($C, { apiKey: '...' })
      - pattern: |
          const $A = await $B($C, { vapidKey: '...' })
  message: Firebase API key found. While this may often be used for analytics only, it is still worth checking for the data leak implications.
  languages: [ts]
  severity: WARNING
```

☑ Active

Delete rule | Gen with AI

Upload

To run the scan, choose the respective scanner with My Rules tag: